

Response to the Operations Research Referee

Manuscript: Accepted Service Adaptation: Compact Allocation Quotients and Minimal Price Memory

Revision: R30, September 23, 2026

Report answered: Price-state R29 report of September 23, 2026.

Review commit: d1f7f977132e46ac9ca582a1d78f5219365509c8.

Reviewed scientific baseline: 51552e4198f09b82c764956b92d7b1ee2ad57aff.

Overview of the revision

We thank the referee for identifying the direct tree-allocation connection and for distinguishing the substantive finite-price result from the earlier nominal revision. We have rebuilt the publication manuscript around the exact laminar mapping, the compact quotient, its rational complexity, and minimal execution memory. The response does not replace a mathematical objection with a change in terminology. The new manuscript contains proofs of the additional results, implements the coefficient-event algorithm needed by the bit bound, and compares three explicitly specified alternatives.

The principal additions are Proposition 2.1 (exact laminar mapping), Theorem 3.1 (occurrence-to-vertex quotient with the finite response closure), Theorem 3.3 (polynomial rational coefficient and preprocessing bounds), Theorem 4.1 (minimal behavioral alphabet, including randomized exact-optimum implementation), Theorem 4.2 (heterogeneous ordered memory frontier), and Proposition 4.3 (outside-option dispersion at a fixed mean payment). Proposition 5.2 completes the endpoint-normal argument for supporting cuts. All previously established mathematics and numerical records remain available in unchanged historical sources and byte-identical predecessor copies.

The new evidence contains 54 completed comparison workers and 12 scaling workers. A separate final audit recompiles all 22 distinct saved certificates from the final code. The largest completed public graph has 2,045 vertices and 8,164 edges; a separate 511-vertex family stores 197,369 response segments. This distinction prevents a low-breakpoint large graph from being presented as a worst-case storage test.

1. Version audit and scientific delta

We agree that the scientifically relevant starting point is the price-state R29 manuscript, not the nominal R29 branch. The source package was reconstructed against the exact upstream tree and commit, and the revised branch descends from that scientific baseline. The current response addresses the subsequent price-state report, including its classical-allocation objection. `PROVENANCE.json` records both distinct source and review commits. The historical report and other revision branches are not edited.

The additional results are identified by theorem rather than by the number of files or pages changed. `NOVELTY_MATRIX.md` separates inherited principles, the R29 theorem, R30 extensions, and unclaimed generalizations.

2. What the central theorem proves

Theorem 3.1 now begins with the occurrence quotient itself. It defines an occurrence, the public vertex, incoming price, relative probability weights, and the normalized conditional optimization. A suffix-path bijection proves equality of the complete continuation problems. Strict concavity then identifies their unique maximizing policies. Only after this equivalence is established do we derive the clipped local response, weighted successor response, and one-sided cap reflection.

Thus the proof no longer moves implicitly from a full history tree to the public graph. The global knot count, total stored segments, and fixed-query execution alphabet are separate conclusions with separate size bounds.

3. Closest literature: tree and laminar allocation

Section 2.2 cites and discusses Mjelde (1983) and Tang (1990), rather than treating chain-nested allocation as the closest class. Proposition 2.1 answers the first question affirmatively: after the positive change of coordinates $z_h = w_h a_{v(h)} x_h$, the history formulation is a separable concave allocation problem with descendant-sum constraints. The objective coefficients, local bounds, continuation caps, and root equality are mapped explicitly.

For the referee's questions about memoization, the answer is also affirmative after normalization. An occurrence response in unconditional allocation units is its reach weight times the conditional public-vertex response. Memoizing the normalized subtree responses gives the same recursion. We do not invent a stochastic obstruction to that equivalence, and we do not claim to originate scalar marginal allocation or memoization.

The quantitative results now concern complete responses on a compact input, coefficient bit heights, and exact execution-state minimization. We prove those statements directly; they are not justified by asserting that no earlier paper could contain a related reduction. The global response-knot bound and the running-maximum label bound follow transparently from the normalized recursion. Their practical execution alphabet can be smaller, and Theorem 4.1 characterizes it.

Tang's publisher abstract gives at most twice the number of variables in one-variable convex subproblems. We report that count as a scalar-subproblem count, not as a complete rational arithmetic bound. We do not transfer ascending-chain complexity bounds to arbitrary branching laminar constraints. Nor do we claim to have established the fastest published exact algorithm for every tree input. Full-text access to all closest classical sources was not obtained in this work session; `LITERATURE_AUDIT.md` states this retrieval boundary explicitly. Consequently, exhaustive historical priority is not marked as independently verified. The mathematical comparisons and new proofs do not depend on inventing an unsupported prior complexity bound.

The broader feasible-set connection is also supplied. EC Section EC.1 proves that translating lower bounds gives a laminar polymatroid and that the root equality selects a base of a truncated rank function. Federgruen and Groenevelt (1986) and Groenevelt (1991) are discussed with their discrete/continuous distinctions intact.

4. Formal compact-DAG quotient and input model

Theorem 3.1 states that any two occurrences of the same public vertex at the same incoming scalar price have identical normalized continuation plans. The proof uses an explicit suffix-history bijection and preserves probabilities, discounts, local rewards, intervals, and caps. The ancestor path can affect the price, but cannot affect the conditional problem after that price is specified.

The statement distinguishes public vertices N , public edges M , total rational encoding length L , and unfolded occurrences H . It gives arithmetic work $O((N + M)N \log(N + 1))$, scalar storage $O(N^2 + M)$, and the bit model in Theorem 3.3. An explicit-tree implementation must read its H occurrence records. This is a comparison of representations, not an impossibility theorem for other compact-graph methods. A normalized memoized classical implementation receives the same quotient bound.

5. Arithmetic operations versus bit complexity

We have taken the stronger of the referee’s two suggested approaches. Theorem 3.3 supplies a conservative polynomial bit bound under an explicit binary rational encoding model. Slopes and intercepts are merged directly as coefficient events. New cap crossings determine intervals but are not fed back into coefficient interpolation.

On any response interval, every coefficient is a sum of source terms multiplied by acyclic path probabilities and discounts. A common denominator divides a fixed product of primitive numerators and denominators raised to an order- N power. The normalization bounds total path weight by the number of visited vertices. This gives bit height $S = O(NL + \log(N + M + 1))$ and the conservative preprocessing bound $O(AS^3)$ for the stated arithmetic count A . This is not a strongly polynomial claim.

EC Section EC.4 separately bounds execution and aggregate certificate heights. Summing rewards across distinct prices can create larger denominators than those in a single response coefficient. Tables 3 and 4 report measured coefficient sizes and separate construction, inversion, execution, and audit times. The comparison does not treat exact Fraction arithmetic as constant cost.

6. Memory theorem and architecture accounting

Theorem 4.1 is no longer restricted to the constructed renewal family. At each public vertex it equates two reached prices precisely when they induce the same local action and the same behavior classes at every successor. Backward induction makes this equivalent to equality of the full future action plan. The smallest common writable alphabet equals the largest number of distinct behavior classes at any public vertex. A constructive transition rule attains the bound. The monotonicity of all future actions also shows that classes are contiguous in the reached-price order.

An explicit exact example demonstrates strict improvement over counting numerical prices: two different terminal prices need only one symbol because their complete behavior is identical. The input graph and price/threshold/action tables are read-only storage, not free total implementation storage. Current public vertex and date are observed. Any retained promise, previous tier, branch identity, or random seed used to distinguish histories counts as writable state unless explicitly included in the public state. Root-price

recomputation is outside the fixed-query alphabet result and is measured separately.

7. Service-contract implications

Proposition 4.3 supplies a comparative static tied to continued participation rather than to a change in mean resource availability. In the renewal family, hold the public process, branch probabilities, and mean payment fixed, and spread continuation caps around their common mean. With a common service-benefit component, this changes outside-option dispersion at fixed mean outside option. For every insufficient deterministic alphabet, its optimal value loss is strictly increasing along this dispersion path.

At exactly zero dispersion one symbol suffices. Any positive dispersion of distinct branch caps requires distinct terminal actions for exact optimality, although the value loss tends to zero as dispersion vanishes. Thus exact implementability and the economic value of additional retention need not move on the same scale. The statement is proved for the specified family; we do not present a synthetic parameterization as a calibrated service application.

8. Zero-release shared table

Section 5 keeps the globally shared-table formulation as a supporting comparator. Proposition 5.1 has public-graph conditional-payment equations and one decision per information cell. The table is not reselected independently in successor subproblems. Its convexity is recognized as standard, rather than advertised as the principal innovation.

The number of information cells is now C_{info} , the restricted value is K^{tab} , and the minimal execution alphabet is K_* . The former conflicting use of K has been removed from the new manuscript. The longer switching/shared-table machinery remains available in the retained theory volumes.

9. Fixed positive-release oracle versus outer design

Proposition 5.2 states an exact value and global supporting inequality for a fixed table center and release radius. The same finite-price compiler handles the resulting local intervals. The supporting inequality is a standard dual-sensitivity construction with the correct weighted corridor multipliers.

The manuscript does not give the continuous outer shared-table design problem the preprocessing guarantee of the fixed-query compiler. No finite exact bound on the number of outer Benders cuts is established or reported as established. This distinction is explicit in the main text and response matrix, while the positive fixed-query result remains intact.

10. Algorithmic comparisons and scale

10.1. Tree allocation. `baselines.py` implements an independent occurrence-indexed scalar marginal reduction. It constructs the unfolded laminar input and forms curves by interpolation without importing the quotient compiler, without memoization, and without receiving an optimum from the quotient method. All eight comparison instances agree in exact rational value. This is a faithful implementation of the stated tree marginal reduction, not a claimed reproduction of unpublished code or a line-by-line reimplementations of

Mjelde or Tang. A normalized memoized implementation would be algorithmically equivalent to the quotient recursion.

10.2. Generic convex optimization. The unfolded quadratic program is solved with SciPy’s sparse trust-constr method, analytic gradient, diagonal Hessian, continuation rows, root equality, and box bounds. All eight final runs report success. The maximum absolute value discrepancy is approximately 3.64×10^{-7} ; the maximum primal residual is approximately 2.62×10^{-14} . Actual stopping settings, iterations, statuses, software versions, and raw repetitions are included. We do not call a floating-point optimizer an exact rational certificate. This is an open-source general convex optimization baseline, not an OSQP or commercial-solver run.

10.3. Promise-state recursion. An independently implemented primal promise-grid recursion operates on the compact public graph. Successor promises use grids of resolutions 10, 20, 40, and 80; the current control absorbs the exact remaining promise. Returned policies are replayed with rational arithmetic. On six cases, the finest-grid maximum feasible loss is approximately 9.77×10^{-5} . This compares discretized primal states with finite exact dual closure without unfolding the public process. It does not claim that an arbitrary uniform grid attains exact optimality.

10.4. Policy graphs and decomposition. The preceding baseline is an actual policy-graph state method. We explain why it is not an SDDP benchmark: the present output specification is an exact single-query rational certificate, whereas general sampled decomposition uses tolerance and sampling choices. There is no relative speed claim about an unimplemented SDDP method. The published policy-graph literature remains credited for avoiding naive scenario enumeration.

10.5. Larger public graphs. The scale study goes to 2,045 vertices, rather than stopping at 190. The largest late-renewal instance has 8,164 edges, 22,419 stored segments, coefficient height 2,213 bits, and a minimal four-symbol execution alphabet. Its three-run median construction time is about 2.616 seconds in this environment. A separate 511-vertex breakpoint-dense family stores 197,369 segments; it prevents the largest graph’s small global knot universe from disguising response-table growth.

Construction, root inversion, certificate generation, audit, machine minimization, 101-query inversion batches, peak process RSS, and raw repetitions are separately recorded. Comparison phases have three repetitions. Scale certificate/audit phases have one explicitly labeled diagnostic run. Censored pilot attempts are retained outside the completed timing tables. Peak RSS includes imports and is not mislabeled as incremental algorithm storage. These measurements establish a specified comparison, not universal dominance or a dedicated-hardware performance claim.

11. Exponential history size

The main narrative no longer uses the number of histories as a surrogate for outperforming modern stochastic programming. Section 2 explains that both public-graph state methods and normalized tree memoization exploit recombination. The residual exactness statement is the finite response closure, the binary-input construction, and the minimal execution machine. The experimental comparator includes a primal state method using the same compact public graph.

12. Focus without deleting historical results

The new main manuscript is 22 PDF pages including its title page, references, and four tables; the new electronic companion is 10 pages. The principal proofs appear in the main manuscript. Optional implementation and boundary details appear in the companion.

No historical derivation was removed to achieve this focus. The R29 main source/PDF and companion source/PDF have byte-identical copies in `predecessor/`; all R1–R29 source and result directories remain. The complete general switching, sensitivity, release, continuous-state, cache, and governance results can therefore still be reviewed, not merely read as a prose summary. `PRESERVATION_REPORT.json` verifies every one of the 1,540 inherited files, allowing changes only to six reader entry points whose originals are retained.

13. Technical details of the finite-price theorem

13.1. Capped versus pre-cap values. The definitions now distinguish H_v , H_v^{pre} , R_v , and d_v . Only the current cap is removed in the pre-cap problem; successor caps stay imposed.

13.2. Root equality and cap. The root has its own cap, and the theorem requires $b_0 \in [L_0, U_0] \subseteq (-\infty, B_0]$. Feasible payment intervals are computed explicitly before the root query.

13.3. Plateaus. The leftmost finite crossing is justified, including a cap at the minimum payment. If two prices produce the same payment, the two optimality inequalities and strict concavity imply the same full optimizer. Therefore dual nonuniqueness does not produce ambiguous actions.

13.4. Occurrence invariance. This is the opening claim and proof of Theorem 3.1, not an unstated induction premise.

13.5. Knot universe versus stored lines. The global union is at most $3N$, while storing a separate response at every vertex can require $O(N^2)$ scalar records. Both bounds remain visible in theory and tables.

13.6. Multiple coordinates. Corollary 3.2 gives at most $2D + N$ response knots but at most $N + 1$ fixed-query price labels. Local clipping knots describe an action response; only a cap barrier changes the stored running price. These are different counts.

The informal $d_v(-\infty)$ notation is replaced by the explicit finite left-tail payment d_v^- .

14. Memory-frontier details

14.1. Controller observations. Section 4.1 specifies the public vertex/date, read-only data, writable register, and transition observations. Carrying a promise, predecessor identity, inherited action, or persistent random seed is not allowed to evade the alphabet count by renaming it. The renewal frontier additionally specifies the common terminal observation and absence of a free branch-revealing physical signal.

14.2. Randomization. Theorem 4.1 extends the necessary alphabet for exact attainment of the full optimum. Conditional averaging preserves the linear feasibility requirements, and strict concavity forces a randomized exact-optimal controller to use the unique action at every positive-probability public history almost surely.

Distinct continuation behaviors therefore need disjoint support in writable symbols. Retained randomness is counted. This argument does not establish equality of every randomized and deterministic restricted-alphabet value; Theorem 4.2 states its frontier for deterministic encodings.

14.3. Broader structure. Theorem 4.1 applies the behavioral quotient throughout the finite-price class. Theorem 4.2 independently extends the ordered renewal frontier to unequal positive branch probabilities, strictly increasing concave terminal rewards, and heterogeneous convex nondecreasing intermediate costs. Exact enumeration compares 2,308 arbitrary, not necessarily contiguous, partitions against the ordered recurrence. The original eight-branch frontier is recovered exactly as a special case.

15. Normal cones at corridor ties and singletons

The proof of Proposition 5.2 and EC Section EC.7 give the complete split. A lower interval normal can be assigned among original lower inequalities attaining the effective endpoint, and an upper normal among original upper inequalities attaining its endpoint. At a singleton both orientations are available. Any nonnegative split preserving the total normal preserves stationarity and complementarity.

The global supporting inequality follows by applying the old multipliers to arbitrary policies feasible under new parameters; it does not presume those policies use the old price machine. Exact tests cover 150 feasible anchors, including fixed tiers, coincident physical/corridor bounds, and zero release width. They contain 96 tied-normal occurrences and 1,185 exact supporting inequalities under physical-first, corridor-first, and equal splits. The statement remains a fixed-query supergradient result, not a finite-iteration outer-design theorem.

16. Literature and positioning

The main references and literature audit now include Mjelde (1983), Tang (1990), Federgruen and Groenevelt (1986), and Groenevelt (1991), followed by the nested-allocation and policy-graph lineages. Proposition 2.1 and EC Section EC.1 map the actual feasible set, rather than compare labels alone. The continuous quadratic, discrete allocation, explicit tree, compact DAG, and general policy-graph input models are not conflated.

The audit records exact bibliographic identifiers and which publisher text was accessible. It does not purport to exclude all possible historical antecedents. Our affirmative case is the stated package of proved compact-response, bit-complexity, minimal-memory, and dispersion results, not an unsupported claim that classical marginal reasoning was absent.

17. Requested ingredients for a compelling resubmission

A. Priority and laminar allocation: exact mapping, normalized memoization equivalence, explicit compact-input results, and broader rank formulation are provided. Exhaustive priority against all full texts is not claimed verified.

B. Complexity: a rational bit theorem replaces reliance on an arithmetic count alone; certificate heights and measured bit growth are separate.

C. Baselines: independent tree reduction, sparse general convex optimization, and a public-graph promise-state method are implemented and reported. SDDP is discussed without invented timings or dominance

claims.

D. Memory: the exact minimal behavioral machine is characterized for the finite-price class; the ordered frontier is generalized beyond the original equal-probability quadratic family.

E. Focus: a 22-page main manuscript and 10-page companion concentrate on the new center, while all broader derivations remain accessible in full.

F. Operational implication: fixed-mean outside-option dispersion strictly raises the value loss from insufficient retention in the stated renewal family, with an exact-alphabet discontinuity at zero dispersion.

18. Presentation and remaining minor points

The title refers to compact allocation quotients and minimal price memory. The abstract says that the finite alphabet belongs to a fixed-query implementation; it does not imply a finite price set for all possible root promises. The abstract is text only and has 191 words. The introduction has no equations or mathematical notation. The manuscript uses author–year citations, an alphabetical bibliography, 11-point text, one-and-a-half spacing, one-inch margins, and no footnotes. The submission checklist distinguishes formatting checks from author declarations that cannot be inferred from repository files.

The prior adverse cache result remains explicit: 83 of 84 strict-tolerance queries refreshed, without a measured speed advantage. The exact verifiers are described as implementation and certificate checks, not as independent mathematical proofs. Source-preservation logistics appear in the evidence documents rather than replacing scientific argument.

19. Concluding response to the editor

The revised paper accepts the correct classical allocation connection and proves the quantitative and operational statements on which its contribution now rests. In addition to the R29 finite response construction, it supplies an explicit quotient proof, polynomial rational preprocessing, a minimal execution-alphabet theorem, a heterogeneous memory frontier, a fixed-mean dispersion comparative static, and a complete tied-bound supporting-cut argument. The computational study now contains actual algorithmic alternatives and distinct scale families with auditable limitations.

We submit these results for renewed mathematical and editorial scrutiny. We do not equate a passing code audit with publication readiness, nor mark an unperformed external literature verification as complete. The complete prior theory and evidence are retained so that the next referee can evaluate the scientific changes directly against the reviewed baseline.